Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země

Karolína Drápelová, Ondřej Kusák

Geometrické vyhledávání bodu

Algoritmy počítačové kartografie

Akademický rok 2025/2026

Praha, 2026

1 Zadání

Nad polygonovou mapou implementuje Ray Crossing Algorithm pro geometrické vyhledávání incidujícího polygonu obsahujícího zadaný bod q.

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

Krok	**Hodnocení**
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.	*+10b*
Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.	*+5b*
*Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.**	*+2b*
Zvýraznění všech polygonů pro oba výše uvedené singulární případy.	*+3b*
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	*+10b*
Řešení pro polygony s dírami.	*+10b*
*Načtení vstupních dat ze *.shp.*	*+10b*
Max celkem:	**65b**

**ošetřeno jen u Winding Number*

2 Vypracování

2.1 Point Location Problem

Point Location Problem patří mezi základní úlohy výpočetní geometrie a v praxi se s ním setkáváme pokaždé, když klikneme myší do interaktivní mapy nebo herního prostředí. Cílem je pro daný bod P v rovině R efektivně určit, ve které konkrétní oblasti se bod nachází.

Zatímco triviální řešení hrubou silou by vyžadovalo testování bodu proti každému polygonu zvlášť, pokročilé algoritmy využívají sofistikované datové struktury k tomu, aby odpověď nalezly v logaritmickém čase. Tento problém je klíčový pro geografické informační systémy (GIS) a počítačovou grafiku. Pro řešení tohoto problému existuje řada algoritmů.

Jednu skupinu tvoří algortimy Half-plane test a Ray Crossing algorithm, které se využívají při řešení s konvexními mnohouhelníky. Pro nekonvexní mnohouelníky se využívají algoritmy Ray Crossing a Winding Number.

2.2 Ray Crossing Algorithm

Ze zkoumaného bodu q je vyslána polopřímka (paprsek) r. Zda bod leží uvnitř či vně polygonu závisí na počtu průsečíků (k) paprsku r s hranami polygonu P. Zda bod leží v polygonu zjistíme jednoduše vydělením počtu vrcholů dvěma. Pokud hodnota po dělení je rovna jedné, bod leží uvnitř polygonu. Pokud je rovna nule, bod neleží v polygonu.

$$k\%2 = \begin{cases} 0 & q \in P \\ 1 & q \notin P \end{cases}$$

Singulárním případem tohoto algoritmu je, když bod q leží na hraně polygonu. Pro tuto variantu je nutné použít dva paprsky - levostranný a pravostranný a udržovat si počet levostranných a pravostranných průsečíků (k_l, k_r). Hlavní myšlenkou je, že pokud $q \in P$, pak $k_l \neq k_r$.

Pseudokód Ray Crossing

Algorithm 1 Ray Crossing ($P = \{p_1, \ldots, p_n\}, q$)

```
Inicializuj k = 0                                          ▷ Počet průsečíků
for každý bod p_i ∈ P do
    x'_i = x_i − x_q
    y'_i = y_i − y_q
    if (y'_i > 0 ∧ y'_{i−1} ≤ 0) ∨ (y'_{i−1} > 0 ∧ y'_i ≤ 0) then   ▷ Vhodný segment
        x'_m = (x'_i y'_{i−1} − x'_{i−1} y'_i)/(y'_i − y'_{i−1})     ▷ Vhodný průsečík
        if x'_m > 0 then
            k = k + 1
if (k%2) ≠ 0 then
    return q ∈ P
else
    return q ∉ P
```

2.3 Winding Number Algorithm

Algoritmus Winding Number je založen na principu pozorovatele, který stojí v testovaném bodě q. Pokud bod $q \in P$ a pozorovatel se chce podívat na všechny vrcholy $p_i \in P$, musí se kolem své osy otočit o celkový úhel 2π. Pokud bod $q \notin P$ je tento celkový úhel menší než 2π, jelikož průvodiče z bodu q mění své směry a tím vznikají úhly se záporným znamínkem.

Výpočet Winding Number se rozumí

$$\Omega(q, P) = \frac{1}{2\pi} \sum_{i=1}^{n} \omega(p_i, q, p_{i+1}).$$

Neboli suma všech rotací ω_i, které musí průvodič opsat nad všemi body $p_i \in P$.

Pro určení orientace úhlů je potřeba sestrojit přímku $p(p_i, p_{i+1})$, která rovinu rozděluje na dvě poloroviny - levou σ_l a pravou σ_r. Pro zjištění, v jaké polorovině se bod q nachází je potřeba určit hodnotu t.

$$t = \begin{vmatrix} x_{i+1} - x_i & y_{i+1} - y_i \\ x_q - x_i & y - y_i \end{vmatrix}$$

Když $t > 0$ bod q leží v levé polorovině σ_l a přičítá se kladný úhel $+\omega_i$. Pokud $t < 0$ bod q leží vpravé polorovině σ_r a přičítá se záporný úhel $-\omega_i$.

$$t = \begin{cases} > 0 & q \in \sigma_l \\ < 0 & q \in \sigma_r \end{cases}$$

Pro výpočet samotných ω_i úhlů je potřeba definovat vektory $\vec{v_1}$ a $\vec{v_2}$, kde

$$\vec{v_1} = (p_i - q)$$

$$\vec{v_2} = (p_{i+1} - q),$$

které vstupují do vztahu

$$\omega_i = \arccos\left(\frac{\vec{v_1} \cdot \vec{v_2}}{|\vec{v_1}| \cdot |\vec{v_2}|}\right)$$

Po sečtení všech úhlů všech vrcholů polygonu a následném vydělením 2π získáme finální výsledek Ω:

$$\Omega(q, P) = \begin{cases} = 1 & q \in P \\ = 0 & q \notin P \end{cases}$$

Pseudokód Winding Number

Algorithm 2 Winding Number

```
Inicializuj Ω = 0, tolerance ϵ                    ▷ Suma všech úhlů
for každou trojici p_i, q, p_{i+1} do
    urči polohu q (q ∈ σ_l ∨ q ∈ σ_r)
    urči úhel ω_i
    if q ∈ σ_l then                               ▷ Bod leží v levé polorovině
        Ω = Ω + ω_i
    else
        Ω = Ω − ω_i                               ▷ Bod leží v pravé polorovině
if ||Ω| − 2π| < ϵ then
    return q ∈ P
else
    return q ∉ P
```

Singulární případy u Winding Number

Prvním ze singulárních případů je, když bod leží ve vrcholu polygonu. Tato podmínka je splněna právě tehdy, když

$$q = p_i.$$

```
for i in range(n):
    #Check if the point is located on a vertex
    if (pol[i].x() == q.x()) and (pol[i].y()== q.y()):
        return index
```

Kód 1: Ošetření v kódu

Druhým singulárním případem je, když bod leží na hraně polygonu, to nastane v případě, když

$$t = 0$$

a zároveň skalární součin vektorů $\vec{v_1}$ a $\vec{v_2}$ bude menší než nula

$$\vec{v_1} \cdot \vec{v_2} < 0.$$

```
#point on the edge
    if sigma == 0 and dot_product < 0:
        return index
```

Kód 2: Ošetření v kódu

2.4 Bod uvnitř min-max boxu

Pro optimalizaci výpočetního času a snížení zbytečných iterací napříč všemi hranami složitých polygonů byla implementována metoda tzv. Bounding Boxu, neboli min-max box. Princip spočívá v tom, že pro daný polygon se nejprve naleznou extrémní souřadnice jeho vrcholů (minimální a maximální hodnota x a y).

Pokud analyzovaný bod q leží striktně mimo tento hraniční obdélník, je geometricky nemožné, aby ležel uvnitř samotného polygonu. V takovém případě se výpočetně náročnější algoritmy (Ray Crossing nebo Winding Number) vůbec nespouští a cyklus přejde k dalšímu polygonu. Tato metoda výrazně zrychluje celkovou analýzu, což je klíčové zejména

při zpracování velkých geografických datasetů (např. mapy států) s tisíci vrcholy.

```
# Urceni hranicniho obdelniku
min_x, max_x, min_y, max_y = self.getBoundingBox(outer_pol)

# Rychla kontrola, zda bod nelezi zcela mimo obdelnik
if not (min_x <= q.x() <= max_x and min_y <= q.y() <= max_y):
    continue
```

Kód 3: Filtrování polygonů pomocí min-max boxu

2.5 Řešení pro polygony s dírami

Reálná geografická data (např. státy obsahující jezera nebo enklávy jiných států) často netvoří jednoduché polygony, ale tzv. komplexní polygony. Ty se skládají z jednoho vnějšího obrysu a jedné nebo více vnitřních děr. Aby výpočetní algoritmy fungovaly korektně, byla datová struktura polygonu v aplikaci upravena na slovník obsahující klíče `outer` (vnější hranice) a `holes` (seznam vnitřních hranic tvořících díry).

Úprava algoritmu Ray Crossing

V případě algoritmu Ray Crossing je adaptace na komplexní polygony poměrně přímočará. Všechny hrany vnějšího polygonu i všech jeho děr se sloučí do jednoho společného seznamu. Paprsek, který protne vnější hranici polygonu, vstoupí do jeho plochy (počet průsečíků je lichý). Pokud následně protne i hranici díry, z platné plochy polygonu vystoupí (počet průsečíků se stane sudým). Výsledná podmínka vyhodnocující počet průsečíků ($k\%2 == 1$) tak matematicky spolehlivě funguje i pro libovolný počet děr.

```
# Spojeni vnejsiho obrysu a vsech der do spolecneho listu
all_inst = outer_pol + holes

# Algoritmus iteruje pres vsechny hrany (obrys i diry)
for pol in all_inst:
    # ... (vypocet pruseciku)
```

Kód 4: Zpracování děr v algoritmu Ray Crossing

Úprava algoritmu Winding Number

U algoritmu Winding Number probíhá ověření ve dvou na sebe navazujících krocích. Nejprve se standardním způsobem vypočítá suma úhlů pro vnější obrys polygonu. Pokud je výsledkem $\Omega = 0$, bod leží mimo vnější obrys a algoritmus ihned končí s negativním výsledkem.

Pokud se však bod nachází uvnitř vnějšího obrysu ($\Omega \neq 0$), je nutné iterovat přes všechny definované díry daného polygonu a aplikovat Winding Number na každou z nich. Pokud algoritmus potvrdí, že bod leží uvnitř jakékoli z těchto děr, je finálně vyhodnocen jako bod ležící mimo platnou plochu.

```
is_in_hole = False

# Dodatecna kalkulace pro vsechny diry polygonu
for hole in holes:
    # ... (vypocet Winding Number pro konkretni diru)

    # Pokud bod lezi uvnitr diry, nelezi v polygonu
    if abs(abs(omega_hole) - 2 * pi) < epsilon:
        is_in_hole = True
        break

if not is_in_hole:
    return index
```

Kód 5: Ověření polohy uvnitř díry pro Winding Number

2.6 Načtení dat ze .shp

Načtení dat se provádí pomocí knihovny *shapefile*. Z důvodů různých souřadnicových systémů (souřadnice obrazovky jsou jiné než souřadnice shapefilu) bylo potřeba shapefile přemapovat do souřadnic obrazovky. Z původních shapefile souřadnic byly spočítány minima a maxima x-ových a y-ových souřadnic. Z těchto údajů pak byly spočítany rozměry.

```
#Calculate the dimensions
      shp_width = glob_max_x - glob_min_x
      shp_height = glob_max_y - glob_min_y
```

Kód 6: Rozměry shapefile států Evropy.

Ty poté byly vycentrovány na střed obrazovky, zároveň byl zvolen 5 % okraj, aby se mapa nedotýkalo okrajů, bylo zvoleno i měřítko.

```
    #Leave a 5% margin so the shapefile doesn't touch the edge
      margin = 0.05
      usable_width = canvas_width * (1 - 2 * margin)
      usable_height = canvas_height * (1 - 2 * margin)

      #Scale
      scale_x = usable_width / shp_width
      scale_y = usable_height / shp_height
      scale = min(scale_x, scale_y)

      #Center the map on the canvas
      x_offset = (canvas_width - (shp_width * scale)) / 2
      y_offset = (canvas_height - (shp_height * scale)) / 2
```

Kód 7: Nastavení pozice mapy na obrazovce.

Následně byly body tvořící původní polygony přemapovány do souřadnic obrazovky a tyto přemapované body vytvořily polygony, které jsou poté načítány v samotné aplikaci.

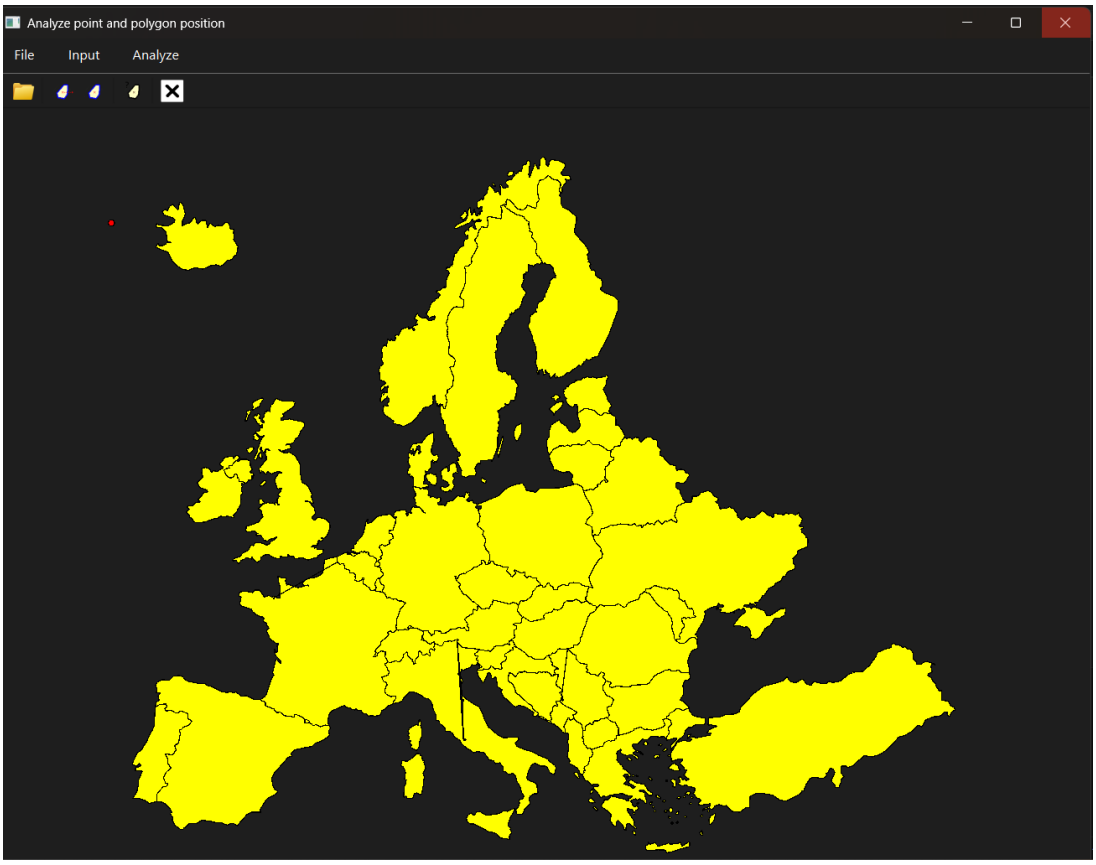
```
 #Transform all coordinates and create QPolygonFs
    self.__polygons = []

    for points in raw_polygons:
        scaled_points = []
        for x, y in points:
            #Shift to 0,0 > scale > add offset
            screen_x = (x - glob_min_x) * scale + x_offset

            #Y-axis is inverted. Maps go up, screens go down.
            screen_y = canvas_height - ((y - glob_min_y) * scale +
y_offset)

            scaled_points.append(QPointF(screen_x, screen_y))

        self.__polygons.append(QPolygonF(scaled_points))
```

Kód 8: Přemapování souřadnic bodů/polygonů.

2.7 Vstupní data

Vstupními daty jsou shapefile soubory evropských zemí, které byly upraveny v GIS software. Byly smazány menší ostrovy a zámořské části Francie a Španělska a kvůli své rozlehlosti i Rusko. Data jsou volně dostupná na stránkách Eurostatu.

Výstupem je aplikace vyvtořená v rozhraní QT, ve které lze provádět analýzy polohy bodu a polygonu. Uživatel načte potřebná data a zvolí bod, výsledkem je zvýraznění polygonu, ve kterém se bod nachází.



Obrázek 1: Printscreen vytvořené aplikace s načtenými daty.

2.8 Dokumentace

2.8.1 Třída Algorithms

Třída *Algorithms* zahrnuje metody *getBoundingBox*, *getPointPolygonPositionRC* a *getPointPolygonPositionWN*. První metoda slouží k určení min-max boxu a vrací minima a maxima x-ových a y-ových souřadnic. Druhá metoda analyzuje vzájemnou polohu polygonu a bodu pomocí Ray Crossing algoritmu a vrací informaci o tom, ve kterém polygonu se bod nachází. Třetí metoda provádí totožnou analýzu, ale používá k tomu Winding Number algoritmus.

2.8.2 Třída Draw

Třída *Draw* obsahuje metody *mousePressEvent* - změna polohy kurzoru myši při kliknutí, *paintEvent* - vykreslení bodu či polygonu, *changeStatus*, *clearData* - vymazání všech polygonů a bodu na obrazovce, *getQ* - vrácení bodu, *getPol* - vrácení polygonů, *setHighlightedPolygon* - zvýraznění polygonu, kterému náleží analyzovaný bod a *LoadShapesToScene* - načtení vstupních dat ve formátu shapefile.

2.8.3 Třída Main Form

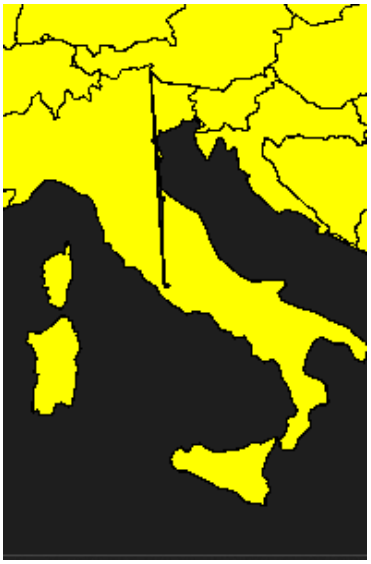
Třída *Main Form* slouží ke spuštění grafického rozhraní aplikace a propojuje jednotlivé metody definované v ostatních třídách.

2.9 Závěr

V rámci této práce byla úspěšně vytvořena aplikace v prostředí QT, která řeší geometrické vyhledávání bodu v polygonu (Point Location Problem). Implementovány byly dva algoritmy: Ray Crossing Algorithm a Winding Number Algorithm. Oba algoritmy byly upraveny tak, zpracovávaly nejen jednoduché, ale i komplexní polygony s dírami. Pro zvýšení výpočetní efektivity byla implementována metoda min-max boxu (Bounding Box), která výrazně urychluje vyhledávání v rozsáhlých datasetech tím, že vylučuje polygony, v jejichž obdélníkovém opsaném celku se bod q nenachází. Aplikace umožňuje přímé načítání dat ve formátu .shp. Součástí řešení bylo i matematické přemapování souřadnic z geografického systému do souřadnicového systému obrazovky se zachováním aspektu a definovaným okrajem.

Při testování na reálných datech z Eurostatu se u vybraných entit (např. Itálie) projevila grafická chyba v interpretaci polygonu. Jak je patrné z obrázku 2, pravděpodobnou příčinou je buď nekonzistence ve vstupním shapefilu, nebo specifická chyba při transformaci souřadnicových polí u velmi členitých objektů. Tato anomálie však nemá vliv na matematickou správnost vyhodnocení polohy bodu algoritmy RC a WN.

Námětem na vylepšení by mohlo být vyskakovací okno zobrazující atributová data (název země) při kliknutí.



Obrázek 2: Printscreen chyby při načítání shapefile Evropy.

3 Literatura a zdroje

BAYER, T. (2008): Algoritmy v digitální kartografii. Nakladatelstvtví Karolinum, Praha.

BAYER, T. (2026): Point Location Problem. Přednáška. Katedra aplikované geoinformatiky a kartografie. Přírodovědecká fakulta UK (cit. 10. 3. 2026). Dostupné z: https://web.natur.cuni.cz/bayertom/images/courses/Adk/adk3_new.pdf.

EUROSTAT (2024): GISCO: Countries. (cit. 7. 3. 2026). Dostupné z: https://ec.europa.eu/eurostat/web/gisco/geodata/administrative-units/countries.

LAWHEAD, J. (2025): Python Shapefile Library. Dostupné z: https://pypi.org/project/pyshp/.